AddressSanitizer (ASan) Beginner's Guide

Catch memory bugs in C/C++ automatically, with every flag and report line explained

1. What ASan is for

ASan is built into the compiler. You add one flag, rebuild, and your program checks every memory access while it runs. When it finds a bug it stops immediately and prints the exact line, the type of bug, and where the memory was allocated. You do not step through anything.

It catches:

- **Heap buffer overflow**: reading or writing past the end of a `new[]` or `malloc` block
- **Stack buffer overflow**: going past the end of a local array
- **Global buffer overflow**: the same, for global arrays
- **Use-after-free**: touching memory after `delete` or `free`
- **Double free** and **invalid free**
- **Memory leaks** (via LeakSanitizer, built in on Linux)

Compared to GDB: with GDB you step through and hunt for the bug yourself. With ASan the program tells you where the bug is, usually at the first bad access. That is why sanitizers give the most bug-catching per minute of effort.

2. Setup and the compile command

ASan works with GCC and Clang on Linux and macOS. On Windows, use WSL2 (Ubuntu) and install with `sudo apt install g++`.

```
g++ -fsanitize=address -g -O0 -fno-omit-frame-pointer main.cpp -o main
./main
```

- `-fsanitize=address`: turns ASan on. Use it when compiling AND linking.
- `-g`: line numbers and function names in the report
- `-O0`: no optimization, so the bug cannot be optimized away while you learn (`-O1` is also fine)
- `-fno-omit-frame-pointer`: keeps stack traces accurate
- `./main`: run normally. No special launcher is needed.

Costs: roughly 2x slower and 2x to 3x more memory. Fine for development and testing; do not ship ASan builds to users.

3. Practice program

Save as `main.cpp`. Each function has one deliberate bug; choose which to run from the command line.

```
#include <cstdlib>
#include <iostream>

void heap_overflow() {
    int* a = new int[3]{10, 20, 30};
    int total = 0;
    for (int i = 0; i <= 3; i++) {   // bug: should be i < 3
        total += a[i];
    }
    std::cout << total << "\n";
    delete[] a;
}

void stack_overflow() {
    int data[3] = {1, 2, 3};
    int idx = 3;
    data[idx] = 99;                  // bug: valid indices are 0..2
    std::cout << data[0] << "\n";
}

void use_after_free() {
    int* p = new int(42);
    delete p;
    std::cout << *p << "\n";         // bug: p is freed
}

void leak() {
    int* p = new int[100];           // bug: never deleted
    p[0] = 1;
}

void double_free() {
    int* p = new int(1);
    delete p;
    delete p;                        // bug: freed twice
}

int main(int argc, char** argv) {
    int which = (argc > 1) ? std::atoi(argv[1]) : 1;
    switch (which) {
        case 1: heap_overflow();  break;
        case 2: stack_overflow(); break;
        case 3: use_after_free(); break;
        case 4: leak();           break;
        case 5: double_free();    break;
    }
    return 0;
}
```

Build once, then run each case:

```
g++ -fsanitize=address -g -O0 -fno-omit-frame-pointer main.cpp -o main
./main 1
./main 2
./main 3
./main 4
./main 5
```

Without ASan, cases 1 to 3 often appear to work and silently corrupt memory. That is what makes these bugs dangerous.

4. Reading an ASan report

Run `./main 1`. You will see something like this (addresses and wording vary by compiler version):

```
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x60300000001c
READ of size 4 at 0x60300000001c thread T0
    #0 0x5581a2c3f2a0 in heap_overflow() main.cpp:7
    #1 0x5581a2c3f5d3 in main main.cpp:46

0x60300000001c is located 0 bytes to the right of 12-byte region
allocated by thread T0 here:
    #0 0x7f... in operator new[](unsigned long)
    #1 0x5581a2c3f1e9 in heap_overflow() main.cpp:4
    #2 0x5581a2c3f5d3 in main main.cpp:46

SUMMARY: AddressSanitizer: heap-buffer-overflow main.cpp:7 in heap_overflow()
```

Part by part:

- `heap-buffer-overflow`: the type of bug. Read this first.
- `READ of size 4`: the bad access was a 4-byte read (an `int`). A bad write says `WRITE`.
- `#0 ... main.cpp:7`: the exact line of the bad access. Frame #0 is where it happened; higher numbers are callers (like `bt` in GDB).
- `0 bytes to the right of 12-byte region`: your block was 12 bytes (3 ints) and you touched the very next byte. The classic off-by-one.
- `allocated by thread T0 here`: where that block came from (`main.cpp:4`, the `new int[3]`).
- `SUMMARY`: a one-line recap, handy for notes and bug reports.

Reading order for any report: (1) bug type, (2) the first `#0` line in your own code, (3) the allocated or freed traces.

5. What each bug looks like

Run	Bug type in first line	Extra info in the report
`./main 1`	`heap-buffer-overflow`	"N bytes to the right of M-byte region" and an allocation trace
`./main 2`	`stack-buffer-overflow`	Offset in the stack frame and the variables in that frame, e.g. `data`
`./main 3`	`heap-use-after-free`	Three traces: where it was used, where it was freed, where it was allocated
`./main 4`	`detected memory leaks`	"Direct leak of 400 byte(s)", printed when the program exits
`./main 5`	`attempting double-free`	Traces of the second free, the first free, and the allocation

6. Useful options

ASan is configured with the `ASAN_OPTIONS` environment variable, set just before the program:

```
ASAN_OPTIONS=detect_leaks=0 ./main 4
ASAN_OPTIONS=abort_on_error=1 ./main 3
ASAN_OPTIONS=detect_stack_use_after_return=1 ./main
```

- `detect_leaks=0`: turn off leak reports (on by default on Linux)
- `abort_on_error=1`: call `abort()` so you can get a core dump and open it in GDB
- `detect_stack_use_after_return=1`: also catch returning pointers to local variables
- Combine options with a colon: `ASAN_OPTIONS=detect_leaks=0:abort_on_error=1`

Combine with UBSan (the next topic) for more coverage:

```
g++ -fsanitize=address,undefined -g -O1 -fno-omit-frame-pointer main.cpp -o main
```

Combine with GDB: run `gdb ./main`, then `run 3`. ASan stops the program at the error and you can use `bt` and `print` there.

7. Using ASan in your projects

Makefile

```
CXXFLAGS = -std=c++17 -Wall -Wextra -g -O1 -fno-omit-frame-pointer -fsanitize=address
LDFLAGS  = -fsanitize=address
```

CMake

```
add_compile_options(-fsanitize=address -fno-omit-frame-pointer -g)
add_link_options(-fsanitize=address)
```

VS Code (`tasks.json`, in the `args` list)

```
"args": [
  "-fsanitize=address", "-fno-omit-frame-pointer",
  "-g", "-O1",
  "${file}", "-o", "${fileDirname}/${fileBasenameNoExtension}"
]
```

A common habit is to keep two build configurations: a normal one, and a `debug-asan` one that you run your tests under.

8. Limits and gotchas

- **Only finds bugs that actually run.** A code path that never executes can't be checked. Good tests matter.
- **Does not find everything.** It misses uninitialized reads (use Valgrind or MemorySanitizer) and undefined behavior such as signed overflow (use UBSan).

- **Do not run an ASan build under Valgrind.** Pick one or the other.
- **Do not combine with ThreadSanitizer.** They are separate builds.
- **Slower and bigger.** Use it for development and CI, not production.
- **Leak detection on macOS:** Apple Clang usually lacks LeakSanitizer. On Linux it works out of the box.
- **Mixed builds:** make sure every object file uses the same sanitizer flags.
- **High optimization levels** can remove code the compiler proves dead, hiding a toy bug. Use `-O0` or `-O1` while learning.

9. Cheat sheet

What	Command
Build with ASan	`g++ -fsanitize=address -g -O1 -fno-omit-frame-pointer main.cpp -o main`
Run	`./main`
Disable leak check	`ASAN_OPTIONS=detect_leaks=0 ./main`
Abort for core dump / GDB	`ASAN_OPTIONS=abort_on_error=1 ./main`
ASan + UBSan together	`-fsanitize=address,undefined`

Report reading order: **bug type**, then **first `#0` line in your code**, **then** allocated / freed traces**.

10. Practice tasks

1. Run all five cases. For each, note the bug type, the reported line number, and which trace shows where the memory came from.
2. Fix `heap_overflow` (change `i <= 3` to `i < 3`) and confirm the report disappears.
3. In `leak()`, add `delete[] p;` and confirm the leak report disappears. Also try `ASAN_OPTIONS=detect_leaks=0` on the original to see it vanish without a fix.
4. Write your own bug: `std::vector<int> v(3); v[3] = 1;`. Does ASan catch it? (Yes, the vector's buffer is on the heap.)
5. Run case 3 with `ASAN_OPTIONS=abort_on_error=1`, then open it in GDB (`gdb ./main`, `run 3`, `bt`).

11. Troubleshooting

- **No line numbers, only addresses:** you forgot `-g`, or the symbolizer is missing. With Clang, install `llvm` so `llvm-symbolizer` is on your PATH.
- **"ASan runtime does not come first":** usually a linking or LD_PRELOAD issue. Put `-fsanitize=address` on the link step too.
- **Nothing reported but you are sure there is a bug:** the path may not run, or the compiler optimized it away. Rebuild with `-O0`.

- **Very slow or out of memory:** ASan needs extra memory. Reduce test size.
- **Next topic:** UndefinedBehaviorSanitizer (UBSan), with the same workflow.